Proactive Dependency-Aware Autoscaling for Microservices

Load Propagation over the Call Graph and SLO-Constrained Resource Allocation

Presented by

LEUDJEU WOUAPPI Beautrel Horssel

Student ID: 22U2079

Supervised by

Pr. DJAM Xaveria Youh Kimbi

University of Yaoundé I
Faculty of Science
Department of Computer Science

October 24, 2026

Outline

1. The problem encountered: exam results in Cameroon
2. Why reactive autoscaling fails: structural limitations
3. What research has started to propose (GRAF, Erms)
4. The question we ask

The problem encountered: the Baccalauréat results in Cameroon

D-Day.

Hundreds of thousands of candidates connect at the same time.
The official website becomes unreachable.

SMS messages sent to 8070 never arrive.
The servers of the Office du Baccalauréat are saturated.

This is not a raw "capacity" failure: it is a failure of *coordination* between components that do not talk to each other.

What happens, concretely

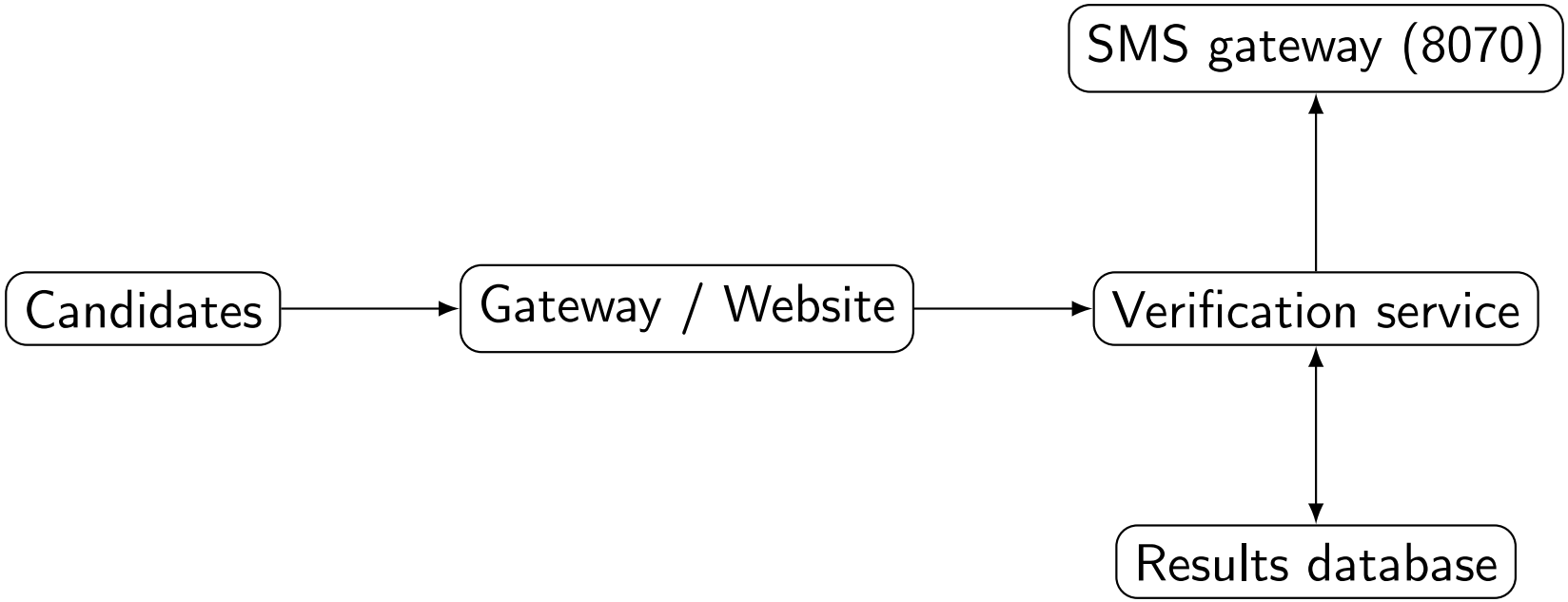
The observed sequence

- Results are published, for example, late in the evening. Between **6 pm and 8 pm**, the consultation platform receives **heavy traffic** and becomes unreachable.
- Candidates retry again and again, which **worsens** the saturation.
- The SMS service (8070) is saturated as well: “phone replies sometimes take a long time to arrive because of the massive congestion of the OBC servers”.
- The authorities recommend trying again “early in the morning or end of the evening”, at less busy hours.

What the user experiences

“I click, the page does not load. I try again. Still nothing. I try the SMS service: the message never arrives. I ask a friend: their site is blocked too.”

Why the website "scales" but the outage persists



- The website (the gateway) can **scale horizontally**: we add instances of the web server.
- But the **real bottleneck** is elsewhere: the database or the SMS gateway, which do not scale at the same pace, or whose **startup time** is long.
- The website's autoscaler **cannot see** that the verification service is saturating, nor that the SMS gateway is congested.
- Each component does its local job "well", but **the chain breaks at the spot nobody monitors**.

Why reactive autoscaling fails: three structural limitations

1. **Late reaction.** The website only reacts once saturation has been observed. Yet at 6 pm, the load is already there. The *cold start* of instances (or the recovery time of a service) adds another 90–150 seconds.
2. **Blindness to dependencies.** The web service has its own autoscaler. The database has its own. The SMS gateway is run by an external operator. **Nobody makes decisions for the whole chain**.
3. **Fixed thresholds.** The website's autoscaler applies a CPU or memory utilization target chosen once and for all, without knowing that the 6 pm–8 pm peak is **predictable** (it happens every year) nor that the real constraint is the end-to-end latency.

Consequence

End-to-end latency depends on *all* components simultaneously, but each autoscaler only sees one isolated component — and sometimes does not even see the dependencies it needs in order to act.

What research has started to propose

GRAF (CoNEXT 2021)

- **Proactive** approach.
- Uses a graph neural network (GNN) to model dependencies.
- Exploits the front-end load to anticipate.
- Goal: minimize total CPU under a latency SLO constraint.

Erms (ASPLOS 2023)

- **Dependency-aware** approach.
- Profiles latency as a function of load and resource usage.
- Determines latency targets for microservices with complex dependencies.
- Reported result: SLA violations reduced by a factor of 5.

These works show that the community has identified the problem. But they leave open the question of the **explicit propagation of the predicted load** over the graph, and of **accounting for the dependencies the autoscaler itself relies on**.

The question we ask

Research question (provisional)

How can we dynamically and simultaneously size all the services of a microservices application, by exploiting the dependency graph and load prediction, so as to meet SLOs at the lowest cost, even when the dependencies required for provisioning itself are not explicitly modeled?

Thank you for your attention.